Instruções compostas

Referência:
https://docs.python.org/pt-br/3.15/reference/compound_stmts.html

8.1. A instrução if

A instrução if é usada para execução condicional:

```
if_stmt: "if" assignment_expression ":" suite
       ("elif" assignment_expression ":" suite)*
       ["else" ":" suite]
```

Ela seleciona exatamente um dos conjuntos *suite* avaliando as expressões *assigment_expression* uma por uma até que uma seja considerada verdadeira (veja a seção Operações booleanas para a definição de verdadeiro e falso); então esse conjunto é executado (e nenhuma outra parte da instrução if é executada ou avaliada). Se todas as expressões forem falsas, o *suite* da cláusula else, se presente, é executado.

8.2. A instrução while

A instrução while é usada para execução repetida desde que uma expressão seja verdadeira:

```
while_stmt: "while" assignment_expression ":" suite
          ["else" ":" suite]
```

Isto testa repetidamente a expressão e, se for verdadeira, executa o primeiro conjunto; se a expressão for falsa (o que pode ser a primeira vez que ela é testada) o conjunto da cláusula else, se presente, é executado e o laço termina.
Uma instrução break executada no primeiro conjunto termina o laço sem executar o conjunto da cláusula else.
Uma instrução continue executada no primeiro conjunto ignora o resto do conjunto e volta a testar a expressão.

8.3. A instrução for

A instrução for é usada para iterar sobre os elementos de uma sequência (como uma string, tupla ou lista) ou outro objeto iterável:

```
for_stmt: "for" target_list "in" starred_expression_list ":" suite
        ["else" ":" suite]
```

A expressão starred_expression_list é avaliada uma vez e deve produzir um objeto iterável. Um iterador é criado para esse iterável. O primeiro item fornecido pelo iterador é então atribuído à lista de destino usando as regras padrão de atribuição (veja Instruções de atribuição), e o conjunto é executado. Isso se repete para cada item fornecido pelo iterador. Quando o iterador está esgotado, o conjunto na cláusula else, se presente, é executado e o laço termina.
Uma instrução break executada no primeiro conjunto termina o loop sem executar o conjunto da cláusula else. Uma instrução continue executada no primeiro conjunto pula o resto do conjunto e continua com o próximo item, ou com a cláusula else se não houver próximo item.

O laço for faz atribuições às variáveis na lista de destino. Isso substitui todas as atribuições anteriores a essas variáveis, incluindo aquelas feitas no conjunto do laço for:

```
for i in range(10):
    print(i)
    i = 5           # não afeta o laço for, porque i será
                    # substituída pelo índice seguinte no range
```

Os nomes na lista de destinos não são excluídos quando o laço termina, mas se a sequência estiver vazia, eles não serão atribuídos pelo laço. Dica: o tipo embutido range() representa sequências aritméticas imutáveis de inteiros. Por exemplo, iterar range(3) sucessivamente produz 0, 1 e depois 2.

Alterado na versão 3.11: Elementos marcados com estrela agora são permitidos na lista de expressões.

8.4. A instrução try

A instrução try especifica manipuladores de exceção e/ou código de limpeza para um grupo de instruções:

```
try_stmt:  try1_stmt | try2_stmt | try3_stmt
try1_stmt: "try" ":" suite
        ("except" [expression ["as" identifier]] ":" suite)+
        ["else" ":" suite]
        ["finally" ":" suite]
try2_stmt: "try" ":" suite
        ("except" "*" expression ["as" identifier] ":" suite)+
        ["else" ":" suite]
        ["finally" ":" suite]
try3_stmt: "try" ":" suite
        "finally" ":" suite
```

Informações adicionais sobre exceções podem ser encontradas na seção Exceções, e informações sobre como usar a instrução raise para gerar exceções podem ser encontradas na seção A instrução raise.

Alterado na versão 3.14: Suporte para a remoção opcional de parênteses de agrupamento ao usar vários tipos de exceção. Veja **PEP 758**.

8.4.1. Cláusula except

As cláusulas except especificam um ou mais manipuladores de exceção. Quando nenhuma exceção ocorre na cláusula try, nenhum manipulador de exceção é executado. Quando uma exceção ocorre no bloco try, uma busca por um manipulador de exceção é iniciada. Essa busca inspeciona as cláusulas except por vez até encontrar uma que corresponda à exceção. Uma cláusula except sem expressão, se presente, deve ser a última; ela corresponde a qualquer exceção.

Para uma cláusula except com uma expressão, a expressão deve ser avaliada como um tipo de exceção ou como uma tupla de tipos de exceção. Parênteses podem ser omitidos se vários tipos de exceção forem fornecidos e a cláusula as não for usada. A exceção levantada corresponde a uma cláusula except cuja expressão é avaliada como a classe ou uma classe base não virtual do objeto exceção, ou como uma tupla que contém tal classe.

Se nenhuma cláusula except corresponder à exceção, a busca por um manipulador de exceção continua no código circundante e na pilha de invocação. [1]

Se a avaliação de uma expressão no cabeçalho de uma cláusula except levantar uma exceção, a busca original por um manipulador será cancelada e uma busca pela nova exceção será iniciada no código circundante e na pilha de chamadas (ela é tratada como se toda a instrução try levantasse a exceção).
Quando uma cláusula except correspondente é encontrada, a exceção é atribuída ao destino especificado após a palavra reservada as nessa cláusula except, se presente, e o conjunto da cláusula except é executado. Todas as cláusulas except devem ter um bloco executável. Quando o final deste bloco é atingido, a execução continua normalmente após toda a instrução try. (Isso significa que, se existirem dois manipuladores aninhados para a mesma exceção e a exceção ocorrer na cláusula try do manipulador interno, o manipulador externo não tratará a exceção.)
Quando uma exceção é atribuída usando as target, ela é limpa no final da cláusula except. É como se

```
except E as N:
   foo
```

fosse traduzido para

```
except E as N:
   try:
      foo
   finally:
      del N
```

Isso significa que a exceção deve ser atribuída a um nome diferente para poder ser referenciada após a cláusula except. As exceções são limpas porque, com o traceback (situação da pilha de execução) anexado a elas, formam um ciclo de referência com o quadro de pilha, mantendo todos os locais nesse quadro vivos até que ocorra a próxima coleta de lixo.
Antes de um conjunto de cláusulas except ser executado, a exceção é armazenada no módulo sys, onde pode ser acessada no corpo da cláusula except chamando sys.exception(). Ao sair de um manipulador de exceções, a exceção armazenada no módulo sys é redefinida para seu valor anterior:

```
print(sys.exception())
None
try:
   raise TypeError
except:
   print(repr(sys.exception()))
   try:
      raise ValueError
   except:
      print(repr(sys.exception()))
   print(repr(sys.exception()))

TypeError()
ValueError()
TypeError()
print(sys.exception())
None
```

8.4.2. Cláusula except*

A cláusula except* especifica um ou mais manipuladores para grupos de exceções (instâncias de BaseExceptionGroup). Uma instrução try pode ter cláusulas except ou except*, mas não ambas. O tipo de exceção para correspondência é obrigatório no caso de except*, portanto, except*: resulta em um erro de sintaxe. O tipo é interpretado como no caso de except, mas a correspondência é realizada com as exceções contidas no grupo que está sendo tratado. Uma exceção TypeError é levantada se um tipo correspondente for uma subclasse de BaseExceptionGroup, pois isso teria uma semântica ambígua.
Quando um grupo de exceções é levantado dentro de um bloco try, cada cláusula except* o divide (consulte split()) em subgrupos de exceções correspondentes e não correspondentes. Se o subgrupo correspondente não estiver vazio, ele se torna a exceção tratada (o valor retornado por sys.exception()) e é atribuído ao alvo da cláusula except* (se houver). Em seguida, o corpo da cláusula except* é executado. Se o subgrupo não correspondente não estiver vazio, ele é processado pela próxima cláusula except* da mesma maneira. Isso continua até que todas as exceções do grupo tenham sido tratadas ou até que a última cláusula except* tenha sido executada.
Após a execução de todas as cláusulas except*, o grupo de exceções não tratadas é mesclado com quaisquer exceções que foram levantadas uma ou mais vezes em cláusulas except*. Esse grupo de exceções, mesclado, se propaga:

```
try:
   raise ExceptionGroup("eg",
      [ValueError(1), TypeError(2), OSError(3), OSError(4)])
except* TypeError as e:
   print(f'caught {type(e)} with nested {e.exceptions}')
except* OSError as e:
   print(f'caught {type(e)} with nested {e.exceptions}')

caught <class 'ExceptionGroup'> with nested (TypeError(2),)
caught <class 'ExceptionGroup'> with nested (OSError(3), OSError(4))
 + Exception Group Traceback (most recent call last):
 |   File "<doctest default[0]>", line 2, in <module>
 |     raise ExceptionGroup("eg",
 |         [ValueError(1), TypeError(2), OSError(3), OSError(4)])
 | ExceptionGroup: eg (1 sub-exception)
 +-+---------------- 1 ----------------
   | ValueError: 1
   +------------------------------------
```

Se a exceção levantada do bloco try não for um grupo de exceções e seu tipo corresponder a uma das cláusulas except*, ela será capturada e encapsulada por um grupo de exceções com uma string de mensagem vazia. Isso garante que o tipo do alvo e seja BaseExceptionGroup de forma consistente:

```
try:
   raise BlockingIOError
except* BlockingIOError as e:
   print(repr(e))

ExceptionGroup('', (BlockingIOError(),))
```

break, continue e return não podem aparecer em uma cláusula except*.

8.4.3. Cláusula else

A cláusula opcional else é executada se o fluxo de controle deixar o conjunto try, nenhuma exceção foi levantada e nenhuma instrução return, continue ou break foi executada. Exceções na cláusula else não são manipuladas pelas cláusulas except precedentes.

8.4.4. Cláusula finally

Se finally estiver presente, especifica um manipulador de "limpeza". A cláusula try é executada, incluindo quaisquer cláusulas except e else. Se uma exceção ocorrer em qualquer uma das cláusulas e não for tratada, a exceção será salva temporariamente. A cláusula finally é executada. Se houver uma exceção salva, ela será levantada novamente no final da cláusula finally. Se a cláusula finally levantar outra exceção, a exceção salva será definida como o contexto da nova exceção. Se a cláusula finally executar uma instrução return, break ou continue, a exceção salva será descartada. Por exemplo, esta função retorna 42.

```
def f():
    try:
        1/0
    finally:
        return 42
```

As informações de exceção não estão disponíveis para o programa durante a execução da cláusula finally.
Quando uma instrução return, break ou continue é executada no conjunto try de uma instrução try...finally, a cláusula finally também é executada "na saída".
O valor de retorno de uma função é determinado pela última instrução return executada. Como a cláusula finally sempre é executada, uma instrução return executada na cláusula finally sempre será a última executada. A função a seguir retorna "finally".

```
def foo():
    try:
        return 'try'
    finally:
        return 'finally'
```